

Weeks 5-6: The Capstone

Document 4 of 6 — Days 29 to 42 (Oct 2 - Oct 15)

Two weeks to build and ship the project that carries your candidacy: an **Expense Management API**, a domain that sounds like real business software to a bank or insurer. You may substitute an inventory, booking, or claims domain — keep the technical requirements identical.

The day changes shape now. Algorithms drop to 45 minutes, learning to 120, building rises to 135, and one hour every day goes to the job search per Document 6. That hour is not optional.

WEEK 5 — Capstone Build and Search Launch

Oct 2 - Oct 8

Day 29 — Friday, Oct 2

Scope the capstone, launch the search

45 min Algorithms	Pattern: 1-D DP (part 3) — partition and counting problems A third 1-D family: the state is 'best answer using the first i elements', and you build it up from smaller prefixes. Solve in this order: 1. Partition Equal Subset Sum (medium) 2. Decode Ways (medium) 3. Combination Sum IV (medium)
120 min Learn	Domain-driven design at a practical level: entities, value objects, aggregates, and aggregate roots. Bounded contexts. Writing a specification before writing code. Package-by-feature versus package-by-layer and the trade-offs of each.
DRILLS · Domain specification and project scaffolding <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a one-page written specification for the Expense API: users, expenses, categories, budgets, the approval workflow, and reporting. Do this before any code. 2. Identify your aggregate roots and write down which entities belong inside which aggregate, and why. 3. Draw the entity relationship diagram on paper, then photograph it for the README later. 4. List every API endpoint you intend to build with its method, path, request shape, response shape, and status codes. Twenty to twenty-five endpoints is the right scope. 5. Create the repository, generate the Spring Boot project, and set up package structure. 6. Write the Flyway baseline migration creating the users table only, and confirm it applies. 7. Write docker-compose.yml with PostgreSQL and confirm the app connects to the containerized database. 8. Add an Actuator health endpoint and confirm it reports the database as up. Commit the running skeleton.	
135 min Build	A clean, running, empty skeleton: Maven project, package structure, Flyway baseline, Docker Compose with PostgreSQL, health endpoint, and a README stub. Committed.
60 min Job search	Document 6, Section 1 — rewrite your resume for backend engineering. Take the full hour. This is the highest-value hour of the week.

✓ **Win of the day: A new project is scaffolded and running, and your backend-focused resume exists for the first time.**

Day 30 — Saturday, Oct 3

Domain model and schema

45 min Algorithms	Pattern: 2-D Dynamic Programming (part 1) — grid paths Two-dimensional DP where the state is a position in a grid. Build the table row by row; each cell depends on the cells above and to the left. Solve in this order: 1. Unique Paths (medium) 2. Minimum Path Sum (medium) 3. Longest Common Subsequence (medium)
120 min Learn	Review schema design decisions: surrogate versus natural keys, soft deletes versus hard deletes, audit columns (createdAt, updatedAt, createdBy), monetary values and why you never use double for money, and time zone handling with timestamptz.
DRILLS · Production-grade schema and audit design <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Create the User, Expense, Category, and Budget entities with correct JPA annotations and relationships. 2. Use BigDecimal for all monetary amounts with explicit precision and scale. Write a comment explaining why double is wrong for money. 3. Add audit fields to every entity using @CreateDate and @LastModifiedDate with @EnableJpaAuditing. 4. Write Flyway migrations for every table with proper constraints, foreign keys, and indexes on the columns you will filter by. 5. Add a CHECK constraint at the database level ensuring expense amounts are positive, and prove it rejects a bad insert. 6. Store all timestamps as timestamptz and write a test proving a value round-trips correctly across time zones. 7. Write a seed migration inserting realistic sample data: 5 users, 8 categories, 60 expenses across three months, and 4 budgets. 8. Verify the whole schema builds from empty with one docker-compose up and one application start.	
135 min Build	Complete, well-designed schema with realistic seed data. Entities mapped and verified. Commit.
60 min Job search	Document 6, Section 2 — write your French-language resume properly, not through a translator.

✓ **Win of the day: Your capstone has a professional schema, and you have a French resume most competitors will not have.**

Day 31 — Sunday, Oct 4

CRUD, DTOs, and the service layer

45 min Algorithms	Pattern: 2-D DP (part 2) — two-sequence problems When the state depends on positions in two different strings or arrays, the table is indexed by both. Edit distance is the canonical example. Solve in this order: 1. Edit Distance (hard) — attempt seriously, this one is worth the time 2. Longest Palindromic Substring (medium) 3. Interleaving String (medium)
120 min Learn	Review the layered patterns from Week 3 and plan today's implementation before writing code. Deliberate design first, then execution — this is how you avoid a mess you have to unwind on Day 41.
DRILLS · Full CRUD with DTOs, validation, and filtering <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write request and response DTOs for every endpoint in your specification, with full validation annotations. 2. Write mappers between DTOs and entities. Keep mapping logic out of controllers and services where possible. 3. Implement the Category CRUD end to end: controller, service, repository, DTOs, validation, exception handling. 4. Implement the Expense CRUD end to end with the same rigour. 5. Add filtering to the expense list endpoint: by date range, category, status, and amount range, all optional. 6. Add pagination and sorting to the expense list endpoint. 7. Wire your GlobalExceptionHandler with custom exceptions for this domain: ExpenseNotFoundException, CategoryInUseException, BudgetExceededException. 8. Test every endpoint manually in Postman and save the collection. 9. Write unit tests for the service layer as you go — do not defer testing to a later day.	
135 min Build	A complete working API surface for expenses and categories, tested manually and covered by unit tests. Commit.
60 min Job search	Document 6, Section 3 — build your 60-company Montreal target list as a spreadsheet.

✓ **Win of the day: Your capstone has a full API surface, and you know exactly which 60 companies you are targeting.**

Day 32 — Monday, Oct 5

Business logic that is not CRUD

45 min Algorithms	Pattern: Greedy — local choices that are provably global Greedy works when you can argue that taking the best immediate option never blocks a better overall outcome. The skill is recognizing when that argument holds. Solve in this order: 1. Maximum Subarray (medium) — Kadane's algorithm 2. Jump Game (medium) 3. Jump Game II (medium) 4. Gas Station (medium) 5. Hand of Straights (medium)
120 min Learn	Designing business rules properly: state machines and legal transitions, invariants that must always hold, and where validation belongs — controller for shape, service for business rules, database for integrity. Guard clauses versus nested conditionals.
DRILLS · Business rules — state machines and budget enforcement <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Design the expense approval state machine on paper: DRAFT, SUBMITTED, APPROVED, REJECTED, REIMBURSED, with the legal transitions between them. 2. Implement the state machine as an enum with a method <code>canTransitionTo(Status target)</code>. 3. Write a service method <code>submitExpense</code> that enforces the transition rules and throws <code>InvalidStateTransitionException</code> otherwise. 4. Implement <code>approve</code> and <code>reject</code>, each checking both the current state and the acting user's role. 5. Implement budget checking: when an expense is submitted, sum approved expenses in that category for the month and reject if the budget would be exceeded. 6. Add a configurable tolerance so budgets can be exceeded by a small percentage with a warning rather than a rejection. 7. Write unit tests covering every legal transition and every illegal one — that is at least fifteen tests. 8. Write a test proving the budget rule triggers exactly at the boundary, and one proving it does not trigger just below. 9. Refactor any nested conditionals in this logic into guard clauses and note the readability difference.	
135 min Build	A working approval workflow and budget enforcement, fully tested. This is the part of the project that is not a tutorial. Commit.
60 min Job search	Document 6, Section 4 — LinkedIn and application tracker set up. Send your first 5 applications.

✓ **Win of the day: Your project contains real business logic no tutorial produced, and your first applications are live.**

Day 33 — Tuesday, Oct 6

Security and JWT authentication

45 min Algorithms	Pattern: Intervals — sorting then sweeping Sort intervals by start time, then walk through merging or counting overlaps. Almost every interval problem is that one move. Solve in this order: 1. Insert Interval (medium) 2. Merge Intervals (medium) 3. Non-overlapping Intervals (medium) 4. Meeting Rooms II (medium)
120 min Learn	Spring Security architecture: the filter chain, authentication versus authorization, SecurityContext, UserDetailsService, PasswordEncoder and BCrypt. JWT structure (header, payload, signature), signing and validation, expiry, and why you never put secrets in the payload. Stateless authentication versus sessions. Role-based access control with @PreAuthorize. CORS configuration.
DRILLS · JWT authentication and role-based authorization <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add Spring Security and observe that every endpoint is suddenly protected by the default login. Note what changed. 2. Implement a UserDetailsService loading users from your database, with BCrypt-hashed passwords. 3. Write a registration endpoint that hashes the password before saving. Verify in the database that no plaintext password is stored. 4. Implement JWT generation on login and a JwtAuthenticationFilter validating the token on every request. 5. Configure the security filter chain: permit registration and login, authenticate everything else, and make the API stateless. 6. Test with curl: call a protected endpoint without a token (expect 401), with an invalid token (expect 401), and with a valid token (expect 200). 7. Add role-based rules so only a MANAGER may approve expenses, using @PreAuthorize. Test with both roles. 8. Add ownership rules so a user can only read and edit their own expenses. Test that user A cannot access user B's expense — this is a real vulnerability class, not a theoretical one. 9. Configure CORS explicitly and explain in a comment what problem it solves. 10. Write integration tests covering authenticated and unauthenticated access to every protected endpoint.	
135 min Build	The API is genuinely secured with JWT, role-based access, and ownership checks, all covered by tests. Commit.
60 min Job search	Send 8 applications, prioritizing Tier 1 companies from your list.

✓ **Win of the day: Your API is properly secured — a strong differentiator at junior level, and a rich source of interview material.**

Day 34 — Wednesday, Oct 7

Reporting and query performance

45 min Algorithms	Pattern: Math & Geometry A small pattern set worth two days, no more. Matrix manipulation and digit arithmetic show up occasionally and are quick wins when they do. Solve in this order: 1. Rotate Image (medium) 2. Spiral Matrix (medium) 3. Set Matrix Zeroes (medium) 4. Happy Number (easy)
120 min Learn	Aggregate queries in JPQL, native queries for reporting where JPQL falls short, database indexing strategy for reporting queries, reading a PostgreSQL EXPLAIN ANALYZE plan (sequential scan versus index scan, estimated versus actual rows), and composite indexes and column order.
DRILLS · Aggregate reporting and query optimization <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Build a monthly-spend-by-category report endpoint returning aggregated totals. 2. Build a budget-utilization endpoint showing spent, remaining, and percentage per category for a given month. 3. Build a top-spenders endpoint aggregating across users, restricted to managers. 4. Build a trend endpoint returning monthly totals for the last twelve months. 5. Seed 100,000 expense rows with a SQL script so your reports run against realistic data volume. 6. Run EXPLAIN ANALYZE on your slowest report query and record the plan and the execution time. 7. Add an appropriate index, rerun EXPLAIN ANALYZE, and record the new plan and time. Write both numbers in your log. 8. Experiment with composite index column order on a two-column filter and record which order performs better and why. 9. Write tests asserting your report figures are correct against known seed data.	
135 min Build	Four working report endpoints, benchmarked and indexed, with before-and-after numbers recorded. Commit.
60 min Job search	Send 8 applications. Begin referral outreach per Document 6, Section 5.

✓ **Win of the day: You measurably made a database query faster and can describe exactly how — a genuine engineering story for interviews.**

Day 35 — Thursday, Oct 8

Testing the capstone properly

45 min Algorithms	Pattern: Review — revisit list clearance Everything you failed on Days 1 to 34 comes back today. Solving a previously-failed problem cold is worth more than solving three new easy ones. Solve in this order: 1. Six problems from your revisit list, timed, blank file 2. Anything still failing goes on the list again for Day 49
120 min Learn	Test strategy for a real application: what to unit test (business rules), what to integration test (wiring and persistence), and what not to test at all (framework behaviour, getters). Test data builders to reduce duplication. Testing secured endpoints with <code>@WithMockUser</code> .
DRILLS · Comprehensive test coverage for the capstone <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a <code>TestDataBuilder</code> class producing valid <code>Expense</code>, <code>User</code>, and <code>Category</code> objects with sensible defaults and overridable fields. 2. Achieve full unit coverage of the approval state machine — every legal and illegal transition. 3. Achieve full unit coverage of budget enforcement, including boundary cases. 4. Write integration tests for every endpoint through <code>MockMvc</code> with authentication, using <code>@WithMockUser</code> for different roles. 5. Write integration tests proving the ownership rules hold: user A cannot read, update, or delete user B's expense. 6. Write repository tests on <code>Testcontainers</code> covering every custom query. 7. Write a test for the full lifecycle: register, log in, create expense, submit, approve, verify state. 8. Run the whole suite and record the count and duration. Aim for 60 or more meaningful tests. 9. Deliberately break three different business rules one at a time and confirm a specific test catches each.	
135 min Build	A serious test suite, all green from one command. This is what separates a portfolio project from a toy. Commit.
60 min Job search	Send 8 applications. Follow up on anything sent 5 or more days ago.

✓ **Win of the day: Your capstone is protected by 60+ meaningful tests, and you cleared your algorithm revisit backlog.**

Day 36 — Friday, Oct 9

Docker and containerization

45 min Algorithms	Pattern: Bit Manipulation The final small pattern set. XOR for finding singles, bit shifting for counting, and masking. Two days total is the correct investment. Solve in this order: 1. Single Number (easy) 2. Number of 1 Bits (easy) 3. Counting Bits (easy) 4. Reverse Bits (easy) 5. Missing Number (easy)
120 min Learn	Docker fundamentals: images versus containers, the Dockerfile instruction set, layer caching and why instruction order matters, multi-stage builds for Java and why they cut image size dramatically, .dockerignore, and choosing a base image. Docker Compose: services, networks, volumes, depends_on, and health checks.
DRILLS · Docker multi-stage builds and Compose orchestration <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a naive single-stage Dockerfile for your app. Build it and record the image size. 2. Rewrite it as a multi-stage build with a JDK build stage and a JRE runtime stage. Record the new size and the difference. 3. Reorder your Dockerfile so dependency resolution happens before source copying. Rebuild twice after a code-only change and observe the cache hit. 4. Add a .dockerignore excluding target, .git, and IDE files. Rebuild and note the reduced build context size. 5. Write docker-compose.yml running the app and PostgreSQL with a named volume, a shared network, and a health check on the database. 6. Add depends_on with a health condition so the app waits for the database to be genuinely ready, not just started. 7. Wipe everything (docker compose down -v), then bring it up from scratch and confirm Flyway builds the schema and the app starts clean. 8. Externalize every secret and connection string as an environment variable with no defaults committed to source. 9. Document the one-command startup in your README and verify it works exactly as written.	
135 min Build	Your entire stack starts from one docker-compose command on any machine. Commit.
60 min Job search	Send 8 applications. Contact 3 Montreal technical recruiters.

✓ **Win of the day: Anyone in the world — including an interviewer — can run your entire application with one command.**

Day 37 — Saturday, Oct 10

Deploy to the cloud

45 min Algorithms	Pattern: Mixed timed set From here your algorithm block is mixed practice. Recognizing which pattern applies from a cold problem statement is now the skill being trained. Solve in this order: 1. Four random mediums from patterns you have covered, 25 minutes each 2. Do not look up which pattern before starting
120 min Learn	Deployment concepts: environment variables and secret management, managed database services, connection pooling with HikariCP and sizing it correctly, health checks and readiness versus liveness, graceful shutdown, and JVM memory settings in a container.
DRILLS · Cloud deployment and production configuration <i>Goal: write every one of these yourself, by hand, no copy-paste.</i> 1. Choose a free-tier host (Render, Railway, or Fly.io) and create an account and a project. 2. Provision a managed PostgreSQL instance and record its connection details as secrets, never in source. 3. Configure a prod profile with production-appropriate settings: no SQL logging, validate-only DDL, tighter connection pool. 4. Deploy the application and get it running against the managed database. 5. Run your Flyway migrations against the production database and confirm the schema built correctly. 6. Test every endpoint against the live URL, including authentication, and fix whatever breaks in the real environment. 7. Configure HikariCP pool size deliberately and write a comment explaining your reasoning. 8. Enable graceful shutdown and confirm in-flight requests complete during a restart. 9. Add the live URL to your README, your resume, and your LinkedIn.	
135 min Build	Your API is publicly deployed and working end to end, with a URL an interviewer can click. Commit.
60 min Job search	Send 8 applications. Add the live demo URL to your resume and LinkedIn.

✓ **Win of the day: Your project has a public URL — very few junior candidates offer that, and it changes how a recruiter reads your resume.**

Day 38 — Sunday, Oct 11

Documentation and presentation

45 min Algorithms	Pattern: Mixed timed set Continue mixed practice. Log which pattern each problem turned out to be and whether you identified it correctly within the first two minutes. Solve in this order: 1. Four random mediums, 25 minutes each 2. Record your pattern-identification accuracy
120 min Learn	OpenAPI and Swagger: annotating endpoints, documenting request and response schemas, documenting error responses, and grouping with tags. What makes a README convince a hiring manager in thirty seconds: the problem, a visual, the stack, and how to run it — in that order.
DRILLS · OpenAPI documentation and a README that sells <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add springdoc-openapi and confirm the Swagger UI loads. 2. Annotate every endpoint with @Operation describing its purpose, and @ApiResponse for each possible status code. 3. Annotate your DTOs with @Schema descriptions and examples so the generated docs are genuinely readable. 4. Configure JWT authentication in the Swagger UI so an interviewer can log in and try endpoints directly in the browser. 5. Write the README: one-paragraph problem statement, architecture diagram, tech stack table, feature list, local setup, live demo link, and API documentation link. 6. Add a 'Design Decisions' section to the README covering three real trade-offs you made and why. 7. Add screenshots of the Swagger UI and a report response to the README. 8. Clean up your commit history — meaningful messages, no 'fix' or 'wip' commits at the tip. 9. Read your own README as if you were a hiring manager with thirty seconds. Cut anything that does not earn its place.	
135 min Build	Fully documented API with a browsable Swagger UI and a README that sells the project. Commit.
60 min Job search	Send 8 applications. Update your resume's Projects section with the finished capstone.

✓ **Win of the day: Your repository now sells itself before you ever speak to anyone.**

Day 39 — Monday, Oct 12

Caching and performance

45 min Algorithms	Pattern: Mixed timed set Mixed practice, weighted toward the patterns you identified slowest yesterday. Solve in this order: 1. Four mediums weighted to your weak patterns 2. One hard, attempted for 30 minutes
120 min Learn	Caching strategy: what is worth caching, cache invalidation as the genuinely hard part, TTL, cache-aside versus write-through, and cache stampede. The Spring Cache abstraction: @Cacheable, @CacheEvict, @CachePut, and cache managers. Redis as a distributed cache and why an in-memory cache breaks when you scale to multiple instances.
DRILLS · Caching with correct invalidation, measured <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add Redis to docker-compose and configure Spring Cache to use it. 2. Write a simple load script (a bash loop with curl, or a small Java class) hitting your slowest report endpoint 100 times, and record the total time. 3. Add @Cacheable to that report method with a sensible key and TTL. Rerun the load script and record the new time. 4. Write both numbers in your log with the percentage improvement — this is an interview story. 5. Add @CacheEvict on the write paths that would invalidate that report. Prove staleness is prevented with a test: read, write, read again, confirm the second read is fresh. 6. Deliberately remove the eviction, reproduce the stale-data bug, then restore it. Note in your log why invalidation is the hard part. 7. Configure different TTLs for different caches and explain your reasoning in a comment. 8. Explain in your log why an in-process cache would break if you ran three instances behind a load balancer.	
135 min Build	Caching implemented with correct invalidation and measured improvement. Commit with the numbers in the message.
60 min Job search	Send 8 applications. Follow up on everything older than 7 days.

✓ **Win of the day: You have hard numbers showing a performance improvement you engineered yourself.**

Day 40 — Tuesday, Oct 13

Observability

45 min Algorithms	Pattern: Mixed timed set Continue mixed practice. Solve in this order: 1. Four mediums 2. Revisit any problem from your list that has failed twice
120 min Learn	Spring Boot Actuator: health, info, metrics, and env endpoints, and securing them. Custom health indicators. Micrometer metrics: counters, gauges, and timers. Correlation IDs and request tracing through logs. Structured JSON logging and why operations teams need it. What to log and what never to log (passwords, tokens, personal data).
DRILLS · Observability — health, metrics, tracing, and audit <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add Actuator and secure its endpoints so only an admin role can reach anything beyond health. 2. Write a custom health indicator reporting the database connection and one reporting Redis. 3. Add a Micrometer counter for expenses created and a timer around your slowest report method. 4. Expose the metrics endpoint and confirm your custom metrics appear. 5. Write a servlet filter that generates a correlation ID per request, puts it in the MDC, and returns it as a response header. 6. Confirm every log line for a single request carries the same correlation ID by tracing one request through the logs. 7. Switch to structured JSON logging and confirm the output is machine-parseable. 8. Audit every log statement in your codebase and remove anything logging a password, token, or full personal record. 9. Add an audit log entry for every approval and rejection, including who acted and when.	
135 min Build	The application is operable and observable: health checks, metrics, correlation IDs, structured logs, and an audit trail. Commit.
60 min Job search	Send 8 applications. Prepare for any screening calls scheduled this week.

✓ **Win of the day: Your application is observable the way senior engineers expect — this is how they evaluate junior work.**

Day 41 — Wednesday, Oct 14

Self code review and refactor

45 min Algorithms	Pattern: Mixed timed set Mixed practice. Solve in this order: 1. Four mediums 2. One hard attempted seriously
120 min Learn	Code review standards: SOLID principles applied concretely rather than abstractly, common code smells (long methods, feature envy, primitive obsession, shotgun surgery), and refactoring patterns (extract method, replace conditional with polymorphism, introduce parameter object). Reading your own code as a hostile reviewer.
DRILLS · SOLID refactoring and self code review <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. List every method in your codebase longer than 25 lines and extract until none remain. 2. Find every instance of duplicated logic and extract it to a shared method or class. 3. Audit access modifiers: make everything as private as it can be, and note anything that was public without needing to be. 4. Find primitive obsession: any place you pass three strings where a small object would be clearer, and fix at least two of them. 5. Apply single responsibility: find one class doing two jobs and split it. 6. Apply open/closed: find one place with a growing if/else chain on a type and replace it with polymorphism or a strategy. 7. Remove every piece of dead code, unused import, and commented-out block. 8. Rename anything vague — no 'data', 'info', 'manager', 'process', or 'handle' without qualification. 9. Run the full test suite after every change and keep it green throughout. If a refactor breaks a test, that is information.	
135 min Build	A clean, reviewed, refactored codebase with all tests still green. Commit.
60 min Job search	Send 8 applications. Review your tracker and calculate your response rate so far.

✓ **Win of the day: Your codebase would show a senior reviewer care and judgement, not just effort.**

Day 42 — Thursday, Oct 15

Learn to present your work

45 min Algorithms	Pattern: Timed mixed review Simulate interview conditions: read the problem, restate it out loud, explain your approach out loud before coding, then code while narrating. Solve in this order: 1. Five problems solved while talking aloud the entire time 2. Record one and watch it back
120 min Learn	Preparing your project narrative: the problem it solves, the architecture and why you chose it, the hardest technical decision and the alternatives you rejected, what broke and how you diagnosed it, and what you would change at ten times the scale.
DRILLS · Presenting your project fluently in two languages <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write out your five-minute project story in full sentences, then reduce it to bullet points you can speak from. 2. Prepare a whiteboard-ready architecture sketch you can draw from memory in two minutes. 3. Prepare specific answers for: why this domain, why PostgreSQL, why JWT over sessions, why you cached that particular query, and how you would scale it to a million users. 4. Prepare your N+1 story, your caching-improvement story, and your indexing story with the actual numbers from your log. 5. Record a five-minute screen-share demo: run the app, walk through Swagger, then walk through the code architecture. 6. Watch it back and note three specific weaknesses in your delivery. 7. Re-record until it is fluent and under six minutes. 8. Rehearse the same walkthrough in French, since a meaningful share of your target employers will interview in French.	
135 min Build	A polished recorded demo and a rehearsed narrative in both languages. Push the final capstone.
60 min Job search	Send 8 applications. You should be near 60 applications sent cumulatively.

✓ **Win of the day: You can present your own work fluently in two languages — the skill that converts a good project into an offer.**